Software Requirements Specification

for

CodeVenture

Version 1.0 approved

Prepared by Braden Shiew, Choy Sheng Jie, Go Wei Zheng, Vincent Tanuwidjaja

Applied_01_MA_G5

18th August 2023

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Revision History **1**
1. Introduction **2**
1.1 Purpose 2
1.2 Intended Audience and Reading Suggestions 2
1.3 Product Scope 2
2. Overall Description **3**
2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 3
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 4
2.7 Assumptions and Dependencies 4
2.8 Stakeholder Analysis 4
3. System Features **13**
3.1 Login System 13
3.2 Menu 14
3.3 Learning Modules 15
3.4 Challenges and Quizzes 16
3.5 Progress Tracker 17
3.6 Stakeholders' Views 17
4. Other Non-functional Requirements **19**
4.1 Performance Requirements 19
4.2 Safety Requirements 19
4.3 Security Requirements 20
4.4 Software Quality Attributes 20
5. Acceptance Criteria **21**
6. Requirement Quality Analysis **23**
7. Appendix A: Glossary **24**

Revision History

Name	Date	Reason For Changes	Version
Choy Sheng Jie	18/8	Starting out working on part 1 and 2	
Choy Sheng Jie	19/8	Worked on Chloe and Richard Robinson's personas	
Vincent Tanuwidjaja	19/8	Created Kim's persona (learner's parent) and started on system feature: Progress Tracker.	
Choy Sheng Jie	20/8	Worked on part 1 and part 2 except 1.2, 2.2, 2.5, 2.6, 2.7 and 2.8. Finished Chloe and Richard Robinson's personas	
Vincent Tanuwidjaja	20/8	Finished all system features in part 3.2, 3.3, 3.4 and 3.5 as well as their functional requirements.	
Go Wei Zheng	20/8	Created John Doe's persona	
Braden Shiew	20/8	Created Shaun's (Educator) persona and worked on 4.1	
Braden Shiew	21/8	Finished parts 4 and 6, modified Glossary	

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Go Wei Zheng	21/8	Started on Stakeholder Analysis (2.8)	
Choy Sheng Jie	21/8	Finished and cleaned up part 1 and 2	
Vincent Tanuwidjaja	21/8	Add two system features, Menu and Stakeholder Views (3.1, 3.6)	
Go Wei Zheng	21/8	Created section 5: Acceptance Criteria	
Choy Sheng Jie	21/8	Added SRS to section 7, minor changes to format	

1. Introduction

1.1 Purpose

The purpose of this Software Requirement Specification is to outline a new project for CodeVenture. CodeVenture is a game made to introduce children or young learners into the world of programming, primarily using Python. This SRS will cover the general scope of the game and outline key concepts for the game. The fundamental features of the game will be covered as well, concerning all fundamental and non-fundamental features of the game.

1.2 Intended Audience and Reading Suggestions

This document is intended for the developers in charge of the production of the application as well as lecturer in charge of assessing this SRS. Section 1 contains the overall purpose and details of this document. Section 2 contains the overall description of this document. Section 3 contains the External Interface Requirements. Section 4 contains System Features of the document while section 5 contains Other Nonfunctional Requirements.

1.3 Product Scope

The purpose of this software is to create a learning infrastructure through a video game concept to encourage young learners to learn programming. It is desired for the software to be a first step for young learners who may be intimidated by the world of programming but want to try it out. The game should be attractive to the relevant generation with simplistic mechanism and simple coding explanations. In the long run, this application may help contribute to society by creating more developers for our advanced, developing world.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

2. Overall Description

2.1 Product Perspective

This product is an independent product and not part of a product family. It is a video game product that has multiple levels to clear, each level being a lesson. To progress further into the game, you must complete the level before it. This game will only teach Python as it is a beginner friendly programming language.

The game will be divided into different sections with missions representing the subtopics of basics to programming. To illustrate, stage 1 may teach players about IDEs and basic Python syntax, while stage 2 may be about datatypes.

In each mission is the lesson, which will contain guides and notes for the uninitiated. Then, after the "teaching" part of the game, the player will be put to test in an exercise which they must clear. It could be in the form of a quiz, a series of QNA, or to put the actual programming into test, a virtual IDE. Each level may not be restricted to just one type of exercise and may have a set time limit depending on the difficulty of the level. After a certain number of levels, players are put to test by having an entire level dedicated to testing their knowledge.

New players who may have experience in coding should be able to start their journey in the game based on their knowledge. For example, if they know a bit of the Python syntax but not data types, they may be able to skip stage 1 of the game.

2.2 Product Functions

The following includes the main functions of the game, which are explored further in section 3.0 of the SRS:

1. The system should a user login system
2. The system should allow a user to enter what they already know
3. The system should give feedback according to user's answers
4. The system should group lesson within a particular subtopic in a section
5. The system should track user's progress and their statistics
6. The system should allow a user to progress as they finish lessons

2.3 User Classes and Characteristics

This application aims to teach young learners who may be in primary school or secondary school the basics of programming, since their school may not provide programming experience. As such, the target audience should be in the age range of 7 to 18. The game must be vibrant and contain elements that may appeal to the generation such as endearingly charming characters or user interface.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

2.4 Operating Environment

For children to play the game, it must aim to meet the hardware requirements of an average smartphone. However, for gamers who might want to enjoy the game on a desktop computer or a laptop, a computer version of the game could be developed as a browser game, or a downloaded game. The general rule of thumb for the game is to not be demanding of processing power.

2.5 Design and Implementation Constraints

The project will be limited to Python as per the assignment requirements. This project should not require a high processing computer to develop. The game shall also not be demanding for devices.

2.6 User Documentation

Since young learners will be playing the game, a manual on how to play the game should be shown when the game is first played and should also be available in the settings section.

2.7 Assumptions and Dependencies

The application is made with teaching beginners in mind, however some may have prior experience to coding before starting the app. Hence, before starting the game the player shall be asked if they have any experience in coding and if so, ask them about their experience in the game.

The game might also be easier to progress if the user is accompanied by a programmer to play the game with. This may affect their gameplay in a positive way. Assuming the programmer guiding has a close relationship with the user, this game may be able to serve a bonding session as well. The user may also be able to learn better with a real-life guide by their side.

2.8 Stakeholder Analysis

Stakeholders:

Young learners: The target audience/end user of the product.

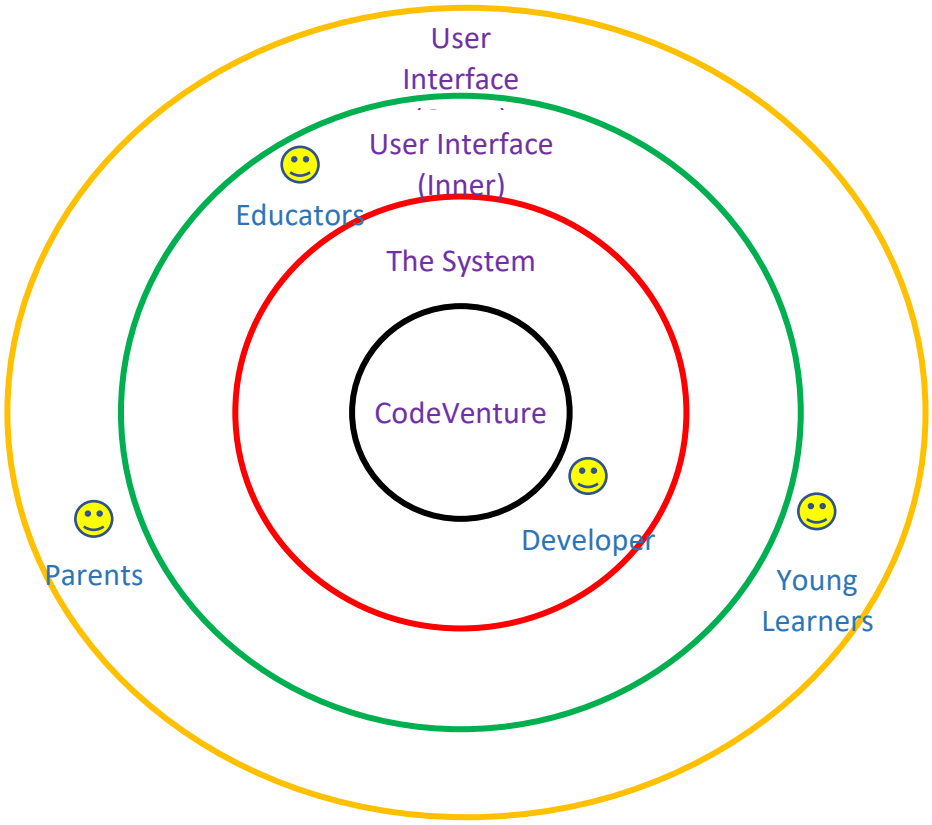
Parents: As a parent of a young learner, he/she is allowed to limit their playtime should the parent deem their child to have played excessively

Educators: Another end user of the product who will assist in the young learners' learning experience

Developer: Person in charge of the process of creating the product

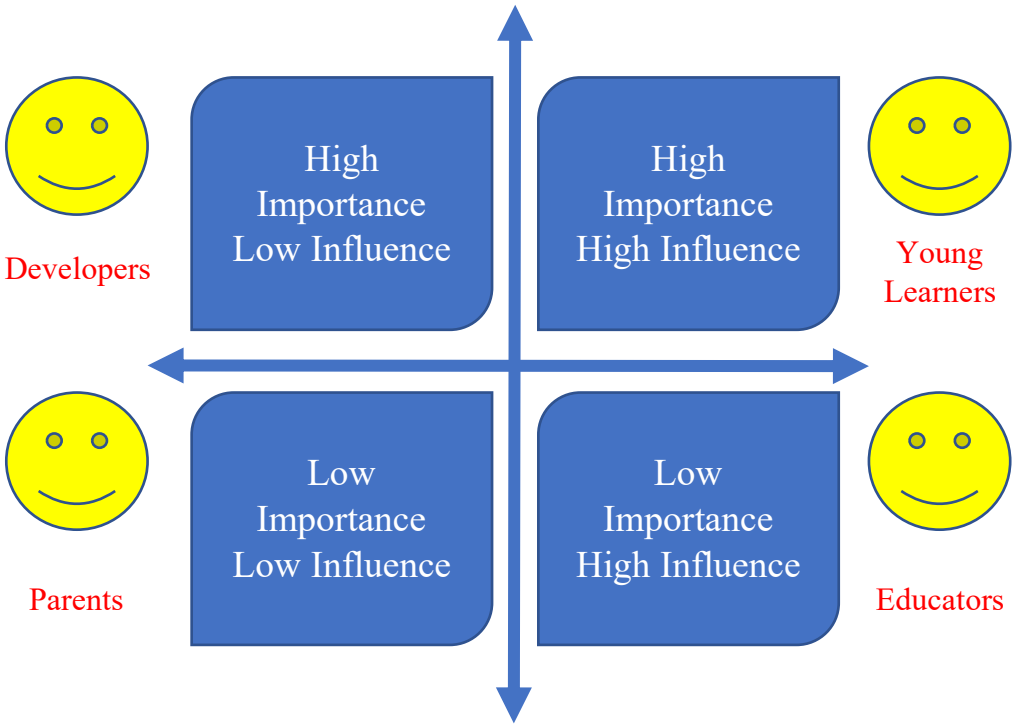
Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Stakeholder Onion Model



Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Stakeholders influence-importance matrix



Rationale/reasoning for why each stakeholder was placed in its respective quadrants:

- Developers
    - While developers are crucial in the process of making the product, their opinion/needs do not really influence it.
- Young Learners
    - As stated in section 1.1, the product is meant to facilitate the process of learning Python programming language. As such, their needs/preference are of utmost importance and would influence how the product is defined.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

- Educators
    - The educators would have to give assignments/homework to the young learners to ensure/ keep track of the students' progress. This would mean we have to define another interface (different from the young learners) to help in their teaching duties.
- Parents
    - As the young learner's parent, the main feature for them would be the **parental controls.** Their opinion/needs are in lower importance/priority and hence, have lower influence in the product creation process.

Personas:

Item	Description
Name	Kim
Picture	
Role	Parent of a young learner
Tagline	"Family woman"
Demographic data	Age 42 and married with a 12-year-old son
Core characteristics	Has passion in technology and for her children's education
Core goals	Provide a supportive environment for her children and encourage their curiosity and enthusiasm for learning
Typical challenges	Unable to find engaging educational resources that are interesting enough for her children

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Singularities	Always participates in parent-teacher meetings, highly supportive of her children's wide range of education
Working situation	Full-time head of HR with flexible working hours
Place of work	Often works remotely at home or in cafes, otherwise in office
Expertise	Proficient in using common office software and familiar with online resources for educational purposes
Main tasks with the system support	Monitor and assess her children's progress through some kind of progress tracker
Most important tasks	Ensure that her children are interested and actively engaging in the program as a part of everyday activities
Least important tasks	Finding out the nitty-gritty details of the program / assignments.
Miscellaneous	Likes to encourage her children to explore various interests from arts, science to tech.

Item	Description
Name	Chloe
Picture	
Role	Young gamer/learner
Tagline	It is what it is
Demographic data	Age 16, Asian
Core characteristics	Enjoys gaming and wants to be a programmer in the future, likes looking at cute things as well.
Core goals	Wants to learn programming

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Typical challenges	There are no computer science or programming subjects taught at school, finds it hard to learn certain concepts from the internet.
Singularities	Desires to be a game programmer
Working situation	High school with several friends
Place of work	High school
Expertise	Time management, as she enjoys playing games while maintaining good grades in school.
Most important tasks	Complete school with a decent grade to study computer science in university
Miscellaneous	Obsessed with anything that may seem cute and bubbly

Item	Description
Name	Richard Robinson
Picture	
Role	Young learner
Tagline	"I find it hard to learn programming even though I have tried starting it a lot of times"
Demographic data	Age 12, American

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Core characteristics	Learns better under guidance by school teachers
Core goals	Wants to be involved with technology in his career
Typical challenges	Difficulty with starting to learn programming
Singularities	Does not have a lot of friends and spends a lot of time studying
Working situation	Has an optimal environment to learn in school and home
Place of work	Primary School
Expertise	Making notes, has notebooks
Most important tasks	Completing schoolwork and studying for tests
Miscellaneous	Likes to be outdoors as it is calming for him. Likes technology products like computers and gadgets.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Item	Description
Name	John Doe
Picture	
Role	Developer
Tagline	"Learning programming can be easier than people think"
Demographic data	• Age 40 • Developer of a program language learning interactive game
Core characteristics	Passionate, Introverted
Core goals	Wants to facilitate the process of learning programming language
Typical challenges	Struggles to speak out due to his introverted nature
Singularities	Critical thinker
Working situation	Medium working load, develops games, software he uses to develop games saves all the program he designed
Place of work	Works mostly at home, occasionally at office due to meetings
Expertise	Mastered the Python programming language
Main tasks with the system support	None (Has to develop the software from scratch)
Most important tasks	Developing the UI and system features of the software
Least important tasks	Adding system features that improves QOL for consumers
Miscellaneous	Enjoys watching video where content creators turn ideas into reality

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Item	Description
Name	Shaun
Picture	
Role	Educator
Tagline	"I've done a lot of dumb things in my life with worse odds."
Demographic data	Age 34, Middle School Teacher
Core characteristics	Athletic, dedicated
Core goals	Pave the road for students' future endeavors
Typical challenges	Struggles to keep students motivated when dealing with mundane tasks
Singularities	Charismatic, articulative
Working situation	7 hours at school, teaches and prepares homework
Place of work	School, home
Expertise	Uses traditional office applications and know programming languages
Main tasks with the system support	Create challenges for students
Most important tasks	Ensuring students are always learning meaningful things
Least important tasks	Excessive administrative duties
Miscellaneous	Really likes rings (he has 10)

Stakeholder interview outcome:

Interviewee: Chloe Tan (Applied_01_MA_G7)

Persona: Young gamer/learner

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Requirements elicited:

- The system shall have a progress tracker
- The system shall have a difficulty system (Learning modules ranked based on their difficulties)
- The user shall be able to unlock aesthetics (Different characters)
- The system shall provide a display of screentime for the user
- The system shall provide user with an optional daily challenge
- The system's learning modules shall be designed so that the user has an intermediate knowledge of Python programming language (e.g. the basics: variable, list, syntax error etc.)

3. System Features

3.1 Login System

3.1.1 Description and Priority

This requirement focuses on user authentication for CodeVenture. Users will need to provide valid details (username and password) to access their accounts. This is of **High** priority.

3.1.2 Stimulus/Response Sequences

1. User runs the application and selects either the login or create account option.
2. The system prompts the user for their username.
3. User inputs their username.
4. The system prompts the user for their password.
5. User inputs their password.
6. The system verifies the user's identity if the user selected login.
7. The system displays the main menu.

3.1.3 Functional Requirements

REQ-1: The system's starting menu shall provide options to select between login and create account where they can choose between parent/educator or learner's account.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

REQ-2: The system shall display a username prompt and wait for user input.

REQ-3: If the user selects the login option and inputs a non-existing username, the system shall display an error message and prompt the user again.

REQ-4: The system shall display a password prompt and wait for user input and afterwards a learner's username if creating a parent account.

REQ-5: Any input that is empty or null should return the user back to the home menu.

REQ-6: If the user selects the login option, the system should authenticate the user details using the existing list of users: Since the username has already been confirmed, compare the existing user's password to the input password.

REQ-7: If successfully authenticated, the user is logged in and therefore the system shall display the menu options.

3.2 Menu

3.2.1 Description and Priority

The Menu feature provides users with a form of navigation through the system. It displays all the next steps the user can take. This feature is of **High** priority as it serves as the central hub for users to access various functionalities.

3.2.2 Stimulus/Response Sequences

1. A young learner logs in to their account.
2. The menu displays all the options available, in this case: All the learning modules, Challenges, Quizzes and Progress Tracker.

3.2.3 Functional Requirements

REQ-8: The system displays the menu in the form of a numbered list.
REQ-9: The menu displays all the options the user can select at the moment.
REQ-10: The system shall prompt the user for a number input, indicating which option they would like to take.
REQ-11: The system shall validate user input to ensure it corresponds to a valid menu option.
REQ-12: An invalid input should display an error message and prompt the user to input again.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

3.3 Learning Modules

3.3.1 Description and Priority

Learning Modules are interactive tutorials that teach young learners various Python programming concepts. This is of **High** priority as these modules are a core feature of CodeVenture, aiming to provide an engaging educational experience.

3.3.2 Stimulus/Response Sequences

1. User logs in to their account.

2. The system displays the menu, including options for the learning modules.

3. User selects a specific module (e.g., Variables).

4. The system displays and presents the interactive module regarding Python functions.

5. User answers an interactive question.

6. The system outputs immediate feedback on whether the answer was correct or not.

7. User completes the module and returns to the menu.

3.3.3 Functional Requirements

REQ-13: The system shall offer a diverse range of learning modules covering various Python programming concepts.

REQ-14: The learning modules should be accessible from the menu and categorized according to the topic and difficulty.

REQ-15: Each learning module shall provide a clear introduction to the concept it covers.

REQ-16: The system shall provide the content in a step-by-step manner, allowing the users to take their time understanding the content and proceeding to the next step on their own will.

REQ-17: Interactive quizzes within modules shall test user understanding and provide immediate feedback.

REQ-18: The system shall track user progress within each learning module, including completed sections and quiz scores.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

REQ-19: The system should always update the progress of the current module within the Progress Tracker, whether it is in progress or completed.

3.4 Challenges and Quizzes

3.4.1 Description and Priority

Challenges and Quizzes are integral components of CodeVenture and is of **High** priority as they supplement the learning modules, offering users the opportunity to apply their newly acquired Python programming knowledge in a practical and interactive manner.

3.4.2 Stimulus/Response Sequences

1. User logs in to their account.
2. The system displays the menu, including options for challenges as well as quizzes.
3. User selects a specific challenge or quiz (e.g., Variables challenge).
4. The system presents a challenge related to variables (e.g., Calculate the area of a rectangle and place it in an appropriate variable).
5. The system evaluates the answer's correctness.

3.4.3 Functional Requirements

REQ-20: The system shall generate programming challenges and quizzes related to each completed learning module.

REQ-21: The system shall allow users to select any challenge or quiz through the menu.

REQ-22: Challenges shall include concise instructions and sample input/output for easy understanding.

REQ-23: Users shall input their solutions directly as Python code within the terminal if it is a challenge, or an option if it is a quiz.

REQ-24: The system shall evaluate the input answer's correctness based on the predefined answer.

REQ-25: Upon receiving incorrect answers, the system shall display a hint and let the user try again.

REQ-26: Upon receiving correct answers, the system shall provide positive feedback and acknowledge the achievement.

REQ-27: After attempting a challenge or quiz, the system shall update the Progress Tracker accordingly.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

3.5 Progress Tracker

3.5.1 Description and Priority

The Progress Tracker is used to monitor and keep track of the user's learning journey within CodeVenture. This is of **medium** priority as it is a crucial part of keeping users motivated and inspired as there is a visualization of their accomplishments and progress but not necessary for their education. For example, the user might be more encouraged if they could see that they are getting closer towards finishing the program.

3.5.2 Stimulus/Response Sequences

1. User navigates to the Progress Tracker from the menu.
2. The system displays all the available modules, challenges as well as quizzes and their progress.
3. User can select a specific module / topic and view the current progress.
4. User can click on the option to find out the next recommended module / challenge.

3.5.3 Functional Requirements

REQ-28: The system shall provide an interface for the user to access the Progress Tracker from the menu.

REQ-29: The Progress Tracker shall provide a complete overview of all the available modules, challenges and quizzes.

REQ-30: The Progress Tracker should display completion status for each module, challenge and quiz, denoted by "COMPLETED", "IN PROGRESS" or nothing if have not started.

REQ-31: Users shall have the option to click on a module, challenge, or quiz to view more detailed progress.

REQ-32: The system shall provide a "Recommended Next Step" based on the user's progress to guide them on their learning journey.

3.6 Stakeholders' Views

3.6.1 Description and Priority

The Stakeholders' Views feature encompasses the different user roles that interact with CodeVenture, providing customized experiences and functionalities based on their roles. This feature is of **High**

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

priority to ensure that learners and parents/educators have appropriate access and information tailored to their needs.

3.6.2 Stimulus/Response Sequences

Learner's View:

1. Learner logs in to their account.

2. The system presents the learner with the main menu, displaying learning modules, challenges, quizzes and Progress Tracker.

3. Learner selects a learning module, completes it, and the progress is updated in the tracker.

Parents/Educators' View:

1. Parent logs in to their account.
2. The system displays the menu, showing the child's achievements and progress.

3.6.3 Functional Requirements

Learners' View:

REQ-33: The system shall provide a menu upon logging in, prominently displaying access to learning modules, challenges, and quizzes.

REQ-34: The system shall showcase their progress tracker, allowing them to monitor their advancements through modules and challenges.

REQ-35: Learners shall have the option to start a new learning module, continue an in-progress one, or review previously completed modules.

Parents/Educators' View:

REQ-36: The system shall display the learner's username upon logging in.

REQ-37: The system shall display the Progress Tracker below the learner's username.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

4. Other Non-functional Requirements

4.1 Performance Requirements

4.1.1 Description

Certain performance requirements should be met to provide the user with satisfaction while playing the game. Fast loading and response times should be a priority as they keep the users engaged. The game should have database read and write operations complete swiftly. Database operations being done swiftly is essential for various functions that will improve user experience such as retrieving user profiles, saving progress and updating leaderboards.

4.1.2 Non-functional Requirements

NFR-1: The system should load within 5 seconds.

NFR-2: If the user performs an action, the system shall respond within 1 second.

NFR-3: The system should have database read and write operations complete within 100 milliseconds.

4.2 Safety Requirements

4.2.1 Description

As the game is mainly targeted towards young learners, it is vital to ensure their safety when playing the game. The collection of personal data from users should be kept to the essentials such as usernames and email addresses. By minimizing data collection, the potential breaches of personal information can be limited. Automated content filtering tools as well as manual review processes should be in place for whenever anything is posted. Content moderation is a must before anything goes public to prevent users from being exposed to inappropriate content or cyberbullying. Similarly, any form of communication features between users should have set phrases banned to prevent inappropriate messages, bullying or sharing of information to occur. Parents should be given the ability to enable parental controls to restrict certain features or to monitor their children’s progress, allowing parents to ensure their children are in a safe environment.

4.2.2 Non-functional Requirements

NFR-4: The system should only collect essential user data.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

NFR-5: If content is posted, the system shall employ content moderation procedures before releasing the post publicly.

NFR-6: If the users engage in chat, the system shall filter certain words or phrases.

NFR-7: Parents shall be given parental controls to monitor or restrict their children's usage of the system.

4.3 Security Requirements

4.3.1 Description

Security is always a concern, but especially so for a game catered for young people. The game should employ strong user authentication measures such as password complexity requirements and the ability to enable two-factor authentication to protect user data and prevent unauthorized access. All user actions should be logged with timestamps so that if any malicious activity occurs, it can be traced back. In the event of data breach, there must be protocols in place to minimize damage and inform users as quickly as possible.

4.3.2 Non-functional Requirements

NFR-8: The system should employ password complexity requirements

NFR-9: The system should allow for two-factor authentication to be enabled

NFR-10: If a user performs an action, the system shall log it accordingly along with a timestamp

NFR-11: If a data breach occurs, the system shall contain the issue and warn its users

4.4 Software Quality Attributes

4.4.1 Description

To ensure the user has fun while learning, it is crucial to ensure the software is of high quality. The game should be supported on major modern devices and browsers including mobile devices, this increases accessibility. As the game is designed for young learners, they should be easily able to navigate and begin coding challenges. The game should display user-friendly error messages whenever errors occur. The error messages should be displayed clearly and promptly so that the issue can be rectified by the user quickly, allowing for a smooth experience. The game should have a failure rate of less than 1% to provide consistent user experience with a reliable system.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

4.4.2 Non-functional Requirements

NFR-12: The system should be supported on major modern devices and browsers.

NFR-13: The system should be easy to navigate even for beginners

NFR-14: If an error occurs, the system shall display a clear error message.

NFR-15: The system should have a failure rate of less than 1%.

5. Acceptance Criteria

Acceptance criteria for REQ_6 – REQ_12:

- The system allows users to create/log in to their account.
- The system provides means of register/logging in when accessing it
- The system prevents unauthorized access if user inputs unregistered credentials
- The system authorizes access to menu for the users when they successfully log in

Acceptance criteria for REQ_13 – REQ_16:

- The system should display a menu as ***UI*** for the users to choose a module of their choice
- The system should provide learning modules with varying difficulties and concepts (e.g. Variables, compile error, syntax etc.)

Acceptance criteria for REQ_17:

- The user should be tested with a quiz after they have supposedly read and understood the module to test their understanding
- The system should provide feedback immediately after they have submitted their answers on the quiz

Acceptance criteria for REQ_18 – REQ_19:

- The system should display a visual presentation of their progress (progress tracker).
- The system should consistently update the progress tracker so that it reflects the user's current work

Acceptance criteria for REQ_20 – REQ_22:

- The system shall generate quizzes related to only modules that the user has completed.
- The system should display a list of quizzes (if prompted by the user) and allow them to choose a quiz of their choice
- The system should provide information and instructions vital to the quizzes presented

Acceptance criteria for REQ_23:

- The user should be able to provide their answer through the system's terminal when attempting a challenge/quiz

Acceptance criteria for REQ_24:

- The system should provide a comparison of the expected output and the output their code generates if it's a challenge.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Acceptance criteria for REQ_25:

- If user answered a quiz/ challenge wrongly, the system should provide a hint should they choose to attempt again.

Acceptance criteria for REQ_26:

- If the user gave a correct answer, the system should provide positive feedback.

Acceptance criteria for REQ_27:

- The system should update the progress tracker showing how much they have done on the particular quiz/challenge after they have attempted it.

Acceptance criteria for REQ_28:

- The system should provide a means to access the progress tracker for detailed information of what they achieved/attempted so far.

Acceptance criteria for REQ_29:

- The system should also provide an overview of available modules, *challenges and quizzes* (as long as it satisfies REQ_20)*.

Acceptance criteria for REQ_30:

- The progress should show whether each attempted module is already completed (denoted by "COMPLETED") or still being attempted (denoted by "IN PROGRESS").

Acceptance criteria for REQ_31:

- When a module/challenge/quiz is selected by user, the system should display detailed progress (What section of the module/challenge/quiz has been attempted)

Acceptance criteria for REQ_32:

- The progress tracker should be utilized to provide a recommend module/challenge for the user.

Acceptance criteria for REQ_33:

- The system should mainly display a menu showing learning modules/challenge/quizzes for users logging in as students.

Acceptance criteria for REQ_34:

- The system should additionally display a progress tracker for students' menu to show their individual accomplishments (completed modules/challenges/quizzes).

Acceptance criteria for REQ_35:

- The system's progress tracker for students should provide new and currently-in-progress modules which allow them to start a new module or continue the module they left off.

Acceptance criteria for REQ_36:

- The system should display their children (users registered as students) username alongside theirs for users logging in as parents.

Acceptance criteria for REQ_37:

- The system should also display their children's progress tracker to allow them to view what their children have been working on.

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

6. Requirement Quality Analysis

Completeness – The requirements are sufficient for the system to satisfy all its objectives (key features are all in Section 3: System Features)

Consistency – The requirements do not contradict each other (the requirements are compatible)

Unambiguity – The requirements are clear (will not be interpreted in another way)

Measurability – The requirements are precise (e.g. *NFR-15:* The system should have a failure rate of less than 1%)

Relevance – The requirements are relevant (the requirements contribute to the satisfaction of the objectives)

Feasibility – The requirements are feasible (can be done with budget, schedule and technology constraints in mind as the requirements are not too complicated to implement)

Comprehensibility – The requirements are comprehensible (the requirements are easy to understand)

Good structuring – The requirements are organized nicely (the requirements are categorized in sections)

Modifiability – The requirements can be modified without affecting others too much

Traceability – The requirements are easily traceable (rationale is clear)

The requirement quality analysis is performed by checking the requirements and evaluating whether they satisfy the requirement quality attributes. As the requirements satisfy the attributes, not much improvement is necessary.

Improvements are shown below:

Old - *NFR-4:* The system should only collect essential user data.

New – *NFR-4:* The system should only collect essential user data, the usernames and email addresses.

Old - *NFR-8:* The system should employ password complexity requirements

New – *NFR-8:* The system should employ password complexity requirements such as requiring capital letters, symbols, numbers and at least 8 characters

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Old - *NFR-13:* The system should be easy to navigate even for beginners.

New – *NFR-13:* The system should have a simple and easy to understand user interface, allowing users to not be overwhelmed.

7. Appendix A: Glossary

REQ: Requirements
NFR: Non-functional requirements
UI: User-interface

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.